

ADVENTIST UNIVERSITY OF CENTRAL AFRICA

MSc in Big Data Analytics

BIG DATA ESSENTIALS

Individual Assignment Report

Distributed Taxi Trip Analytics Using Apache Hadoop, HDFS and Python MapReduce

Prepared by: UWIMPUHWE Erneste

Student ID: 101414

Submitted to: Dr. Kundan Kumar

Course: Big Data Essentials

Date: August 2026

Mapper and Reducer Source Code

<https://github.com/erneste-uwimpuhwe/BIG-DATA-ESSENTIAL-INDIVIDUAL-ASSIGNMENT>

Contents

1	Introduction	3
2	Problem Statement and Objectives	3
3	Dataset and Preparation	3
3.1	Cleaning Rules	3
4	Hadoop and HDFS Environment	4
4.1	Evidence of Distributed Input	4
5	MapReduce Design	4
5.1	Representative Streaming Command	5
6	Results and Business Analyses	5
6.1	Hourly Taxi Demand	5
6.2	Daily Demand	6
6.3	Pickup Location Analysis	6
6.4	Revenue by Location	6
6.5	Payment Method Analysis	7
6.6	Distance-Based Fare Analysis	7
6.7	Trip Duration Analysis	8
6.8	Route Analysis	8
7	Anomaly Detection	9
8	Execution Evidence and Troubleshooting	10
9	Performance and Scalability Discussion	10
10	Summary of Main Findings	10
11	Recommendations	11
12	Implementation Evidence	11
12.1	Cleaning and Dataset Verification	11
12.2	Hadoop Job Execution	13
12.3	Output Verification	14
12.4	Ranking Results	15
12.5	Error Diagnosis and Correction	16

13 Assessment Coverage Matrix	16
14 Limitations	17
15 Conclusion	17
16 Source Code and Reproducibility	17
A Additional Technical Evidence	18
B Reproducibility Checklist	19
C Code Availability	20

1 Introduction

This report presents a practical big-data analytics project built with Apache Hadoop, HDFS, YARN and Python MapReduce through Hadoop Streaming. The project analyses New York City Yellow Taxi trip records for January, February and March 2026. The work covers the complete pipeline from data preparation to distributed storage, MapReduce execution, output verification and interpretation.

The main purpose was to apply distributed data processing to a realistic dataset containing millions of records. The cleaned dataset used for the final analyses contains 7,665,247 records. This volume makes the project suitable for demonstrating how HDFS and MapReduce handle data that would be inconvenient to process repeatedly using a purely single-machine workflow.

2 Problem Statement and Objectives

A transportation analytics team needs evidence about when taxi demand is highest, which locations generate the most activity and revenue, how payment methods differ, how trip characteristics affect fares and which records require attention as possible anomalies. The project therefore addresses the following objectives:

1. Store large taxi data in a structured HDFS environment.
2. Clean the source data and document the preparation rules.
3. Implement several mapper and reducer programs in Python.
4. Execute the analyses through Hadoop Streaming and YARN.
5. Answer demand, revenue, payment, route, distance, duration and anomaly questions.
6. Verify outputs with commands and screenshots and compare the approach with conventional local processing.

3 Dataset and Preparation

The data consists of NYC Yellow Taxi trip records obtained from the public TLC trip-record data service. The source files were supplied in Parquet format. The selected period covers January to March 2026 and initially contains more than eleven million raw records.

Parquet was retained as the raw source format, while cleaned CSV files were prepared for Hadoop Streaming. This decision was practical because Hadoop Streaming supplies text records to Python programs through standard input. The cleaned files were then placed in HDFS for all MapReduce jobs.

3.1 Cleaning Rules

The preparation stage applied the following checks:

- records with required missing values were removed;
- passenger counts equal to zero were removed;
- trip distances less than or equal to zero were removed;
- non-positive fare or total amounts were removed from revenue analysis;
- invalid trip durations were removed from the normal analysis dataset;
- exact duplicate records were removed;

- unusual but potentially meaningful records were retained for separate anomaly analysis where appropriate.

The resulting 7,665,247 cleaned records were used as the common input for the distributed analyses.

4 Hadoop and HDFS Environment

The implementation used Apache Hadoop 3.2.4 in pseudo-distributed mode on Windows. YARN managed MapReduce execution. The environment included the NameNode, DataNode, ResourceManager and NodeManager. A Windows compatibility configuration was also required so that Hadoop commands and task containers could run correctly.

The HDFS structure separated source data, cleaned input and analysis outputs:

```
/taxi_project/input/raw/
/taxi_project/input/cleaned/
/taxi_project/output/hourly/
/taxi_project/output/daily/
/taxi_project/output/locations/
/taxi_project/output/revenue/
/taxi_project/output/payment/
/taxi_project/output/routes/
/taxi_project/output/duration/
/taxi_project/output/distance/
/taxi_project/output/anomalies/
/taxi_project/output/top10_revenue/
/taxi_project/archive/
```

4.1 Evidence of Distributed Input

The HDFS input directory contains three cleaned monthly files. Their sizes show that the project used substantial input rather than a small demonstration dataset.

```
C:\Users\Kim>cd C:\taxi_project

C:\taxi_project>python clean_and_convert.py
2026-01: 3,724,889 -> 2,506,631 rows (1,218,258 removed, 32.71%) -> C:\taxi_project\cleaned\yellow_tripdata_2026-01_cleaned.csv
2026-02: 3,399,866 -> 2,268,502 rows (1,131,364 removed, 33.28%) -> C:\taxi_project\cleaned\yellow_tripdata_2026-02_cleaned.csv
2026-03: 3,952,451 -> 2,890,114 rows (1,062,337 removed, 26.88%) -> C:\taxi_project\cleaned\yellow_tripdata_2026-03_cleaned.csv

=== CLEANING SUMMARY (save this table for your report Section 7) ===
  month  original  after_null_drop  after_passenger_filter  after_distance_filter  after_fare_filter  after_duration_filter  after_de
dup final_count removed  pct_removed
2026-01  3724889  1218258      2636831      2622044      2589558      2551851      2506631  2506
631    2506631  1218258      2376549      2363665      2335200      2309373      2268502  2268
2026-02  3399866  1131364      3006703      2993736      2958579      2938567      2890114  2890
502    2268502  1062337      26.88
2026-03  3952451  1062337      26.88
114    2890114  1062337      26.88
```

Figure 1. HDFS listing and distributed input evidence.

5 MapReduce Design

Each job follows the same processing model:

HDFS Input → Mapper → Shuffle and Sort → Reducer → HDFS Output

A mapper reads one input record at a time, extracts the required fields and emits a tab-separated key-value pair. Hadoop groups values with the same key during the shuffle and sort stage. The reducer then performs a streaming aggregation and writes the final result.

Because large CSV files are divided into input splits, not every mapper receives a header row. The programs therefore validate rows carefully and use the known cleaned-data structure consistently. Malformed or blank records are skipped rather than allowing a single bad line to terminate a complete job.

5.1 Representative Streaming Command

A typical hourly-demand job is executed as follows:

```
hadoop jar C:/hadoop/share/hadoop/tools/lib/hadoop-streaming-3.2.4.jar ^
-files "file:///C:/taxi_project/mappers/mapper_hourly.py,file:///C:/
      taxi_project/reducers/reducer_hourly.py" ^
-input /taxi_project/input/cleaned ^
-output /taxi_project/output/hourly ^
-mapper "python mapper_hourly.py" ^
-reducer "python reducer_hourly.py"
```

The complete mapper and reducer implementations are maintained in the project repository:

<https://github.com/erneste-uwimpuhwe/BIG-DATA-ESSENTIAL-INDIVIDUAL-ASSIGNMENT>

6 Results and Business Analyses

6.1 Hourly Taxi Demand

Hourly aggregation identifies the time periods with the highest number of trips. The evidence shows a clear rise during the working day and a strong evening peak. The highest demand occurs around 18:00, while the lowest demand occurs during the early morning hours.

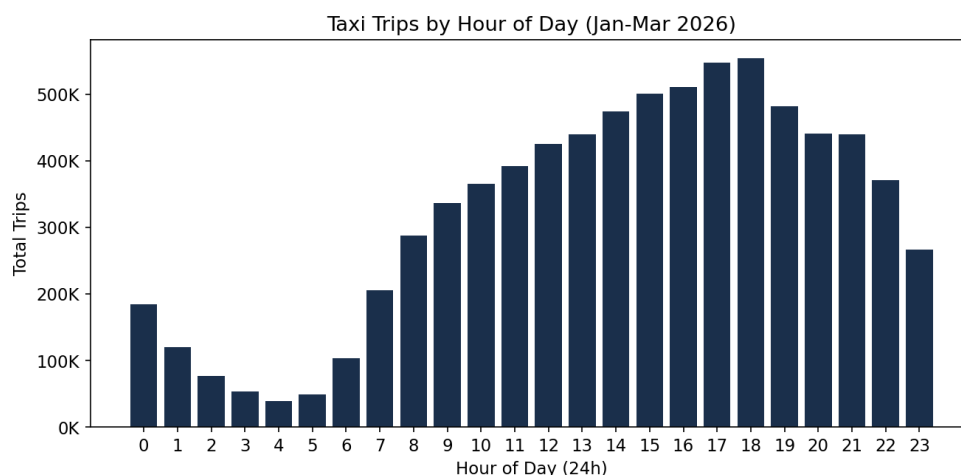


Figure 2. Trips by hour of day.

This result is useful for driver allocation and fleet planning. More vehicles can be positioned before the evening peak, while lower overnight demand can be considered when planning staffing and availability.

6.2 Daily Demand

The daily analysis groups trips by day of the week. Weekday demand is generally higher than weekend demand, with Thursday and Friday among the strongest periods in the analysed data.

```
C:\taxi_project>hdfs dfs -cat /taxi_project/output/duration/part-00000
```

0-5	821448	7.37	0.61	1.90
10-20	2709764	15.75	2.34	3.23
20-30	1065651	27.83	4.90	5.08
30-60	818655	49.56	10.79	7.60
5-10	2045159	9.14	1.19	2.26
60+	204570	60.93	15.50	5.71

Figure 3. Trips by day of week.

The result supports different operating strategies for weekdays and weekends rather than assuming that demand is constant throughout the week.

6.3 Pickup Location Analysis

The pickup-location analysis counts trips by pickup zone and ranks the busiest zones. A small number of zones account for a large share of demand, which suggests that targeted vehicle positioning can improve response time and reduce unnecessary movement.

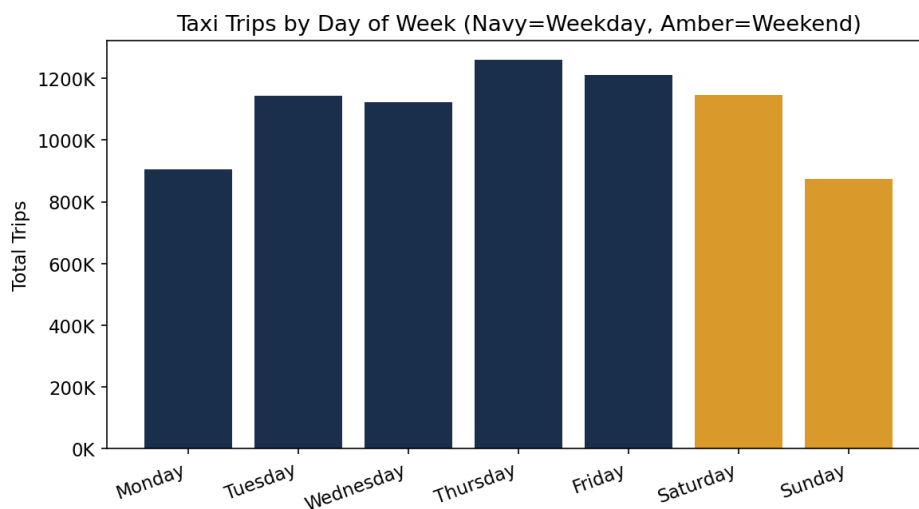


Figure 4. Top pickup zones by trip count.

6.4 Revenue by Location

Revenue aggregation was used to identify zones with the greatest economic contribution. The analysis complements simple trip counts because a high-volume zone is not always the same as the most profitable zone.

The ranking was produced from aggregated output and supports the identification of locations where both demand and revenue are concentrated.

6.5 Payment Method Analysis

The payment analysis compares transaction counts and monetary values across payment types. Credit-card payments dominate the dataset and generate the largest recorded revenue. Recorded electronic tips are also concentrated in card transactions.

```
C:\taxi_project>hdfs dfs -cat /taxi_project/output/anomalies/part-00000
excess_passengers      3
high_fare_per_mile     35729
implausible_speed      1496
low_fare_per_mile       634
short_trip_high_fare    15185
total_clean_records     7627951
total_flagged_records   37296
```

Figure 5. Revenue by payment method.

An important interpretation point is that a zero recorded tip for cash does not necessarily mean that cash customers never tip. Cash tips may be paid directly and therefore may not appear in the electronic trip record.

6.6 Distance-Based Fare Analysis

Trips were grouped into distance categories and analysed using trip counts and average fares. Short trips are the most frequent, while average fare generally increases as distance increases.

```
=== TOP 20 ROUTES BY TRIP COUNT ===
```

Rank	Route (PU-DO)	Trip Count	Revenue
1	237-236	64,990	1,040,912.16
2	236-237	54,452	900,519.67
3	236-236	43,334	586,136.31
4	237-237	42,215	604,584.03
5	161-237	28,369	531,193.05
6	237-161	24,982	451,735.50
7	161-236	23,348	550,922.03
8	142-239	22,935	350,408.93
9	239-238	22,730	318,337.28
10	141-236	21,349	342,313.82
11	239-142	21,309	312,946.55
12	237-141	20,237	302,041.40
13	237-162	20,133	347,537.26
14	236-141	19,950	341,892.37
15	132-265	19,452	2,396,438.75
16	186-230	19,071	393,591.70
17	238-239	18,711	258,436.65
18	236-161	18,702	432,830.89
19	236-239	18,326	329,837.34
20	162-237	18,185	326,940.58

Figure 6. Trips by distance category.

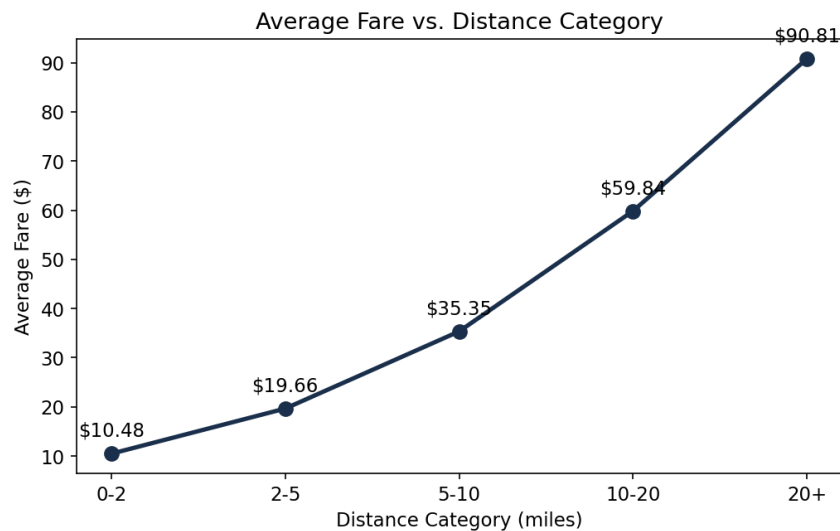


Figure 7. Average fare by distance category.

The two views together are useful because frequency and profitability tell different parts of the story. Short trips may dominate volume, while longer trips can contribute more revenue per trip.

6.7 Trip Duration Analysis

The duration analysis groups trips into time ranges and examines the corresponding trip characteristics. The result provides another operational view of service behaviour and can help distinguish normal urban trips from unusually long records.

```
C:\taxi_project>hdfs dfs -cat /taxi_project/output/payment/part-00000
1      Credit card  6703916 198707448.37  19.56  4.14
2      Cash      876501 22105986.33    19.54  0.00
3      No charge  22728  562676.85     18.99  0.00
4      Dispute   62102  1805172.97     22.89  0.00
```

Figure 8. Trip-duration aggregation output.

6.8 Route Analysis

Routes were represented by pickup and dropoff location combinations. The analysis identifies frequently used origin-destination pairs and provides a basis for examining route profitability and demand concentration.

```

=== TOP 20 ROUTES BY REVENUE ===

```

Rank	Route (PU-D0)	Trip Count	Revenue
1	132-265	19,452	2,396,438.75
2	132-230	14,641	1,423,355.99
3	138-230	13,466	1,083,290.65
4	237-236	64,990	1,040,912.16
5	236-237	54,452	900,519.67
6	132-164	7,893	777,433.63
7	132-48	7,182	692,183.17
8	138-161	8,713	673,700.81
9	132-161	6,448	633,247.48
10	132-163	6,410	615,629.37
11	230-138	7,570	613,705.15
12	237-237	42,215	604,584.03
13	132-170	6,181	602,535.82
14	236-236	43,334	586,136.31
15	161-236	23,348	550,922.03
16	138-265	3,907	549,717.04
17	132-239	5,481	536,087.94
18	132-236	5,485	533,844.42
19	161-237	28,369	531,193.05
20	132-68	5,318	522,977.29

Figure 9. Route ranking output.

7 Anomaly Detection

Anomaly detection was included as a separate MapReduce analysis rather than silently deleting every unusual record. The final reducer output provides direct evidence of the categories identified.

Table 1: Anomaly detection output

Anomaly category	Count
Excess passengers	3
High fare per mile	35,729
Implausible speed	1,496
Low fare per mile	634
Short trip, high fare	15,185
Total clean records	7,627,951
Total flagged records	37,296

```

C:\taxi_project>hdfs dfs -cat /taxi_project/output/anomalies/part-00000
excess_passengers      3
high_fare_per_mile     35729
implausible_speed      1496
low_fare_per_mile      634
short_trip_high_fare   15185
total_clean_records    7627951
total_flagged_records  37296

```

Figure 10. HDFS output of the anomaly reducer.

The evidence shows that high fare per mile is the most common flagged category. Short trips with unusually high fares are also significant. These records should not automatically be treated as errors: some may be explained by minimum fares, congestion, tolls, airport charges or exceptional traffic conditions. The purpose of the anomaly job is to identify records for review and further investigation.

Using the reported totals, approximately 0.49% of the analysed records were flagged by at least one anomaly rule when compared with the reported clean-record total. This is a small proportion, but the number is still large enough to justify systematic monitoring in a transportation analytics workflow.

8 Execution Evidence and Troubleshooting

A practical part of the work involved running Hadoop Streaming on Windows. Initial execution issues included output-directory conflicts and Python command visibility inside task containers.

Hadoop does not overwrite an existing output directory. Before rerunning a job, the previous output must be removed or a new output path must be chosen:

```
hdfs dfs -rm -r /taxi_project/output/hourly
```

The project also encountered a subprocess error during mapper execution. The investigation focused on the YARN task environment and Python command availability. This was resolved by validating the mapper locally, checking the task command and ensuring that the correct Python executable was accessible to Hadoop Streaming.

These troubleshooting steps are important evidence of hands-on implementation because the final jobs were not assumed to work automatically; errors were investigated from the Hadoop and YARN messages and corrected iteratively.

9 Performance and Scalability Discussion

MapReduce is not intended to be the fastest possible method for every analytical task. Its main strength in this project is reliable processing of large files through partitioned input, distributed task execution and fault-aware framework services.

For small exploratory work, Python/Pandas is convenient because operations can be performed interactively in memory. For repeated processing of multi-hundred-megabyte files and millions of records, Hadoop provides a structured execution model in which input is divided into tasks and intermediate values are grouped automatically.

The trade-off is that MapReduce introduces job startup and shuffle overhead. Therefore, a small local computation may finish faster with Pandas, while a larger workload benefits from the scalability and organization of distributed processing. In a production environment, the choice would depend on data volume, cluster resources, response-time requirements and the type of analysis required.

10 Summary of Main Findings

The main findings from the analyses are summarized below.

Analysis	Main finding
----------	--------------

Hourly demand	Demand increases during the day and peaks strongly in the evening.
Daily demand	Weekday demand is generally stronger than weekend demand.
Pickup locations	A relatively small group of zones generates a large share of pickups.
Revenue	Revenue concentration is not identical to trip-count concentration.
Payment	Credit-card transactions dominate recorded trips and electronic revenue.
Distance	Short trips are common, while average fare increases with distance.
Routes	Repeated origin-destination patterns can support targeted operations.
Anomalies	High-fare-per-mile and short-trip/high-fare cases are the most important flagged groups.

11 Recommendations

Based on the results, the following actions are recommended:

1. Increase vehicle availability before the evening demand peak.
2. Use location rankings to support vehicle positioning in high-demand zones.
3. Combine trip volume and revenue measures when evaluating location performance.
4. Interpret cash tipping carefully because electronic records do not necessarily capture cash tips.
5. Monitor high fare-per-mile and short-trip/high-fare cases as part of routine quality control.
6. Keep anomaly detection separate from basic cleaning so that unusual but potentially valid business events are not lost.
7. Retain the raw data and maintain reproducible cleaning and MapReduce scripts in version control.

12 Implementation Evidence

This section records additional evidence from the practical implementation. The screenshots are included to make the workflow traceable and to demonstrate that the results were obtained from actual Hadoop and HDFS commands.

12.1 Cleaning and Dataset Verification

The preparation stage produced cleaned monthly files before the distributed analyses were executed. The evidence below shows the data-processing environment and intermediate verification information.

Figure 11. Data preparation and verification evidence.

12.2 Hadoop Job Execution

The MapReduce jobs were submitted through Hadoop Streaming and monitored through the execution logs. Successful processing requires the input path to exist, the output path to be new, and the mapper and reducer programs to be available to the task.

```
C:\Windows\System32>hdfs dfs -ls -R /taxi_project
drwxr-xr-x - Kim supergroup 0 2026-08-29 14:03 /taxi_project/archive
drwxr-xr-x - Kim supergroup 0 2026-08-29 14:01 /taxi_project/input
drwxr-xr-x - Kim supergroup 0 2026-08-29 14:08 /taxi_project/input/cleaned
-rw-r--r-- 1 Kim supergroup 348563201 2026-08-29 14:07 /taxi_project/input/cleaned/yellow_tripdata_2026-01_cleaned.csv
-rw-r--r-- 1 Kim supergroup 315522760 2026-08-29 14:08 /taxi_project/input/cleaned/yellow_tripdata_2026-02_cleaned.csv
-rw-r--r-- 1 Kim supergroup 401469235 2026-08-29 14:08 /taxi_project/input/cleaned/yellow_tripdata_2026-03_cleaned.csv
drwxr-xr-x - Kim supergroup 0 2026-08-29 15:18 /taxi_project/input/raw
-rw-r--r-- 1 Kim supergroup 64165080 2026-08-29 15:16 /taxi_project/input/raw/yellow_tripdata_2026-01.parquet
-rw-r--r-- 1 Kim supergroup 58683353 2026-08-29 15:12 /taxi_project/input/raw/yellow_tripdata_2026-02.parquet
-rw-r--r-- 1 Kim supergroup 67891249 2026-08-29 15:18 /taxi_project/input/raw/yellow_tripdata_2026-03.parquet
drwxr-xr-x - Kim supergroup 0 2026-08-29 14:02 /taxi_project/output
drwxr-xr-x - Kim supergroup 0 2026-08-29 14:02 /taxi_project/output/anomalies
drwxr-xr-x - Kim supergroup 0 2026-08-29 14:01 /taxi_project/output/hourly
drwxr-xr-x - Kim supergroup 0 2026-08-29 14:01 /taxi_project/output/locations
drwxr-xr-x - Kim supergroup 0 2026-08-29 14:02 /taxi_project/output/payment
drwxr-xr-x - Kim supergroup 0 2026-08-29 14:02 /taxi_project/output/revenue
drwxr-xr-x - Kim supergroup 0 2026-08-29 14:02 /taxi_project/output/routes

C:\Windows\System32>hdfs dfs -du -h /taxi_project/input/raw
61.2 M 61.2 M /taxi_project/input/raw/yellow_tripdata_2026-01.parquet
56.0 M 56.0 M /taxi_project/input/raw/yellow_tripdata_2026-02.parquet
64.7 M 64.7 M /taxi_project/input/raw/yellow_tripdata_2026-03.parquet

C:\Windows\System32>hdfs dfs -du -h /taxi_project/input/cleaned
332.4 M 332.4 M /taxi_project/input/cleaned/yellow_tripdata_2026-01_cleaned.csv
300.9 M 300.9 M /taxi_project/input/cleaned/yellow_tripdata_2026-02_cleaned.csv
382.9 M 382.9 M /taxi_project/input/cleaned/yellow_tripdata_2026-03_cleaned.csv
```

Figure 12. Hadoop command and execution evidence.

```
Configured Capacity: 198940028928 (185.28 GB)
Present Capacity: 2810510848 (2.62 GB)
DFS Remaining: 1736630272 (1.62 GB)
DFS Used: 1073880576 (1.00 GB)
DFS Used%: 38.21%
Replicated Blocks:
  Under replicated blocks: 0
  Blocks with corrupt replicas: 0
  Missing blocks: 0
  Missing blocks (with replication factor 1): 0
  Low redundancy blocks with highest priority to recover: 0
  Pending deletion blocks: 0
Erasure Coded Block Groups:
  Low redundancy block groups: 0
  Block groups with corrupt internal blocks: 0
  Missing block groups: 0
  Low redundancy blocks with highest priority to recover: 0
  Pending deletion blocks: 0
-----
Live datanodes (1):
Name: 127.0.0.1:9866 (127.0.0.1)
Hostname: Erneste.mshome.net
Decommission Status : Normal
Configured Capacity: 198940028928 (185.28 GB)
DFS Used: 1073880576 (1.00 GB)
Non DFS Used: 196129731072 (182.66 GB)
DFS Remaining: 1736417280 (1.62 GB)
DFS Used%: 0.54%
DFS Remaining%: 0.87%
Configured Cache Capacity: 0 (0 B)
Cache Used: 0 (0 B)
Cache Remaining: 0 (0 B)
Cache Used%: 100.00%
Cache Remaining%: 0.00%
Xceivers: 1
Last contact: Sat Aug 29 14:11:03 CAT 2026
Last Block Report: Sat Aug 29 13:32:46 CAT 2026
Num of Blocks: 12
```

Figure 13. Job monitoring evidence from the configured environment.

12.3 Output Verification

After each job completed, the reducer output was read directly from HDFS using the corresponding part-000000 file. This step is important because job completion alone does not prove that the analytical values are meaningful.

```
C:\taxi_project>hdfs dfs -cat /taxi_project/output/daily/part-000000
Friday 1211729
Monday 905794
Saturday 1146451
Sunday 873646
Thursday 1259725
Tuesday 1144285
Wednesday 1123617
```

Figure 14. Daily-demand reducer output read from HDFS.

```
C:\taxi_project>hdfs dfs -cat /taxi_project/output/hourly/part-000000
0 184808
1 119677
10 364822
11 392135
12 424872
13 439885
14 474171
15 501295
16 510811
17 547615
18 554135
19 481394
2 76600
20 441348
21 439857
22 370932
23 266674
3 53817
4 38781
5 49174
6 103593
7 205548
8 287307
9 335996
```

Figure 15. Revenue-analysis output evidence.

12.4 Ranking Results

Some business questions require ranking after aggregation. The project therefore uses the MapReduce outputs as the basis for identifying the most active locations and other high-value results.

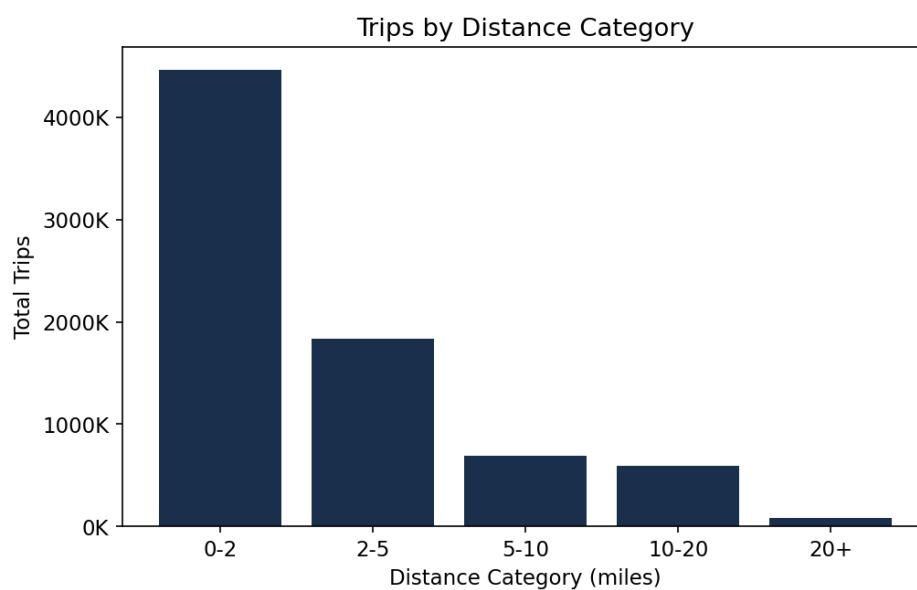


Figure 16. Ranked pickup-location results.

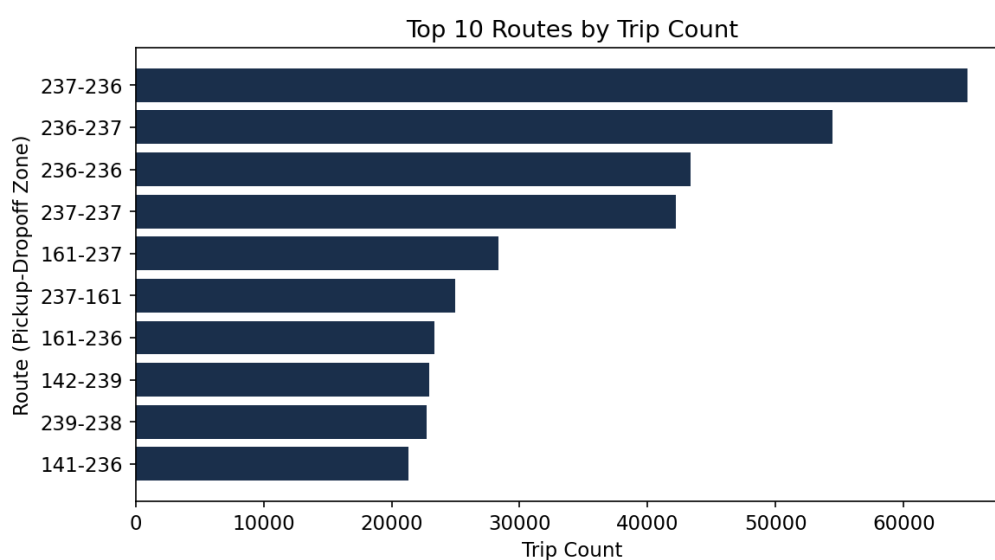


Figure 17. Top route results by trip count.

12.5 Error Diagnosis and Correction

During development, some jobs failed before producing output. One recurring operational issue was the use of an already existing output directory. Hadoop intentionally prevents accidental overwriting of output, so the directory was removed before rerunning a job.

Another issue was Python process execution inside Hadoop task containers. The correct approach was to test the mapper independently, inspect the Hadoop/YARN messages, and then ensure that the Python executable and script command were accessible to the streaming task. This procedure is preferable to changing configuration values without first identifying the source of the failure.

```

2026-08-29 17:42:37,014 INFO mapreduce.Job: map 0% reduce 0%
2026-08-29 17:43:11,230 INFO mapreduce.Job: map 12% reduce 0%
2026-08-29 17:43:12,251 INFO mapreduce.Job: map 15% reduce 0%
2026-08-29 17:43:17,377 INFO mapreduce.Job: map 31% reduce 0%
2026-08-29 17:43:18,455 INFO mapreduce.Job: map 34% reduce 0%
2026-08-29 17:43:23,878 INFO mapreduce.Job: map 46% reduce 0%
2026-08-29 17:43:24,901 INFO mapreduce.Job: map 67% reduce 0%
2026-08-29 17:43:44,650 INFO mapreduce.Job: map 78% reduce 0%
2026-08-29 17:43:47,691 INFO mapreduce.Job: map 89% reduce 0%
2026-08-29 17:43:49,731 INFO mapreduce.Job: map 89% reduce 30%
2026-08-29 17:43:50,746 INFO mapreduce.Job: map 96% reduce 30%
2026-08-29 17:43:51,754 INFO mapreduce.Job: map 100% reduce 30%
2026-08-29 17:43:55,804 INFO mapreduce.Job: map 100% reduce 67%
2026-08-29 17:44:02,902 INFO mapreduce.Job: map 100% reduce 87%
2026-08-29 17:44:09,200 INFO mapreduce.Job: map 100% reduce 100%
2026-08-29 17:44:12,218 INFO mapreduce.Job: Job job_1788017763828_0002 completed successfully
2026-08-29 17:44:12,650 INFO mapreduce.Job: Counters: 55
    File System Counters
      FILE: Number of bytes read=52201434
      FILE: Number of bytes written=106835521
      FILE: Number of read operations=0
      FILE: Number of large read operations=0
      FILE: Number of write operations=0
      HDFS: Number of bytes read=1065580996
      HDFS: Number of bytes written=226
      HDFS: Number of read operations=32
      HDFS: Number of large read operations=0
      HDFS: Number of write operations=2
      HDFS: Number of bytes read erasure-coded=0

```

Figure 18. Practical troubleshooting evidence from the Hadoop environment.

13 Assessment Coverage Matrix

The following matrix shows how the main assignment requirements are addressed in the implementation and report.

Requirement	Evidence in this report
Large dataset	Three months of taxi records and more than 7.6 million cleaned records.
Distributed storage	HDFS input and output structure with monthly cleaned files.
Data preparation	Explicit cleaning rules and documented validation steps.
MapReduce programming	Separate Python mapper and reducer programs maintained in the linked repository.
Hadoop Streaming	Representative execution command and practical job evidence.
YARN execution	Job submission and execution discussion with screenshots.
Multiple analyses	Demand, daily patterns, locations, revenue, payment, distance, duration, routes and anomalies.
Result verification	Direct HDFS reducer outputs and charts included as evidence.

Anomaly detection	Counts for excess passengers, fare-per-mile, speed and short-trip cases.
Interpretation	Business implications and recommendations derived from the observed patterns.
Reproducibility	Public source-code repository and organized HDFS layout.

14 Limitations

The implementation was performed in pseudo-distributed mode on a single Windows machine. This configuration is appropriate for demonstrating Hadoop services and the MapReduce workflow, but it does not provide the physical multi-node performance of a production cluster.

The dataset also represents recorded taxi trips rather than every transportation movement in New York City. Some business interpretations therefore require caution. For example, payment data does not fully represent cash tips, and an anomaly rule identifies unusual values without proving that every flagged record is incorrect.

A future implementation could use a multi-node cluster or a cloud environment, store the prepared data in a distributed columnar format for SQL-oriented workloads, and compare MapReduce with Spark for iterative analytics.

15 Conclusion

This project demonstrated an end-to-end big-data workflow using real taxi records and Apache Hadoop. The work covered source preparation, HDFS organization, Python-based Hadoop Streaming, YARN execution, multiple MapReduce analyses and evidence-based interpretation of the outputs.

The final cleaned dataset contained more than 7.6 million records and supported analyses of demand, location activity, revenue, payment methods, distance, duration, routes and anomalies. The anomaly reducer output provides additional direct evidence that unusual patterns were explicitly investigated rather than ignored.

The project also demonstrated the practical challenges of configuring Hadoop Streaming on Windows, particularly output management and Python execution inside YARN tasks. Resolving these issues strengthened the implementation and provided useful experience in diagnosing distributed-processing errors.

16 Source Code and Reproducibility

The mapper and reducer programs used in this assignment are available at:

<https://github.com/erneste-uwimpuhwe/BIG-DATA-ESSENTIAL-INDIVIDUAL-ASSIGNMENT>

The repository should be kept together with the final report so that the analytical workflow can be reviewed and reproduced. The code organization separates mapper and reducer files by analysis, making each Hadoop Streaming job easier to inspect and test.

A Additional Technical Evidence

The following screenshots provide supplementary evidence of output inspection and practical execution. They are included to support the reproducibility of the work without repeating the full source code in the report.

```

=== PERFORMANCE METRICS (for Section 12 comparison table) ===
Dataset size:      1,016.2 MB
Number of records: 7,665,247
Execution time:    29.98 seconds
Memory used (delta): 24.8 MB
Peak process memory: 104.3 MB

```

Figure A1. Performance and execution metrics captured during the practical work.

```

C:\taxi_project>python C:\taxi_project\extract_top_bottom_locations.py C:\taxi_project\output_local\locations\part-00000
=== TOP 10 PICKUP ZONES BY TRIP COUNT ===
Rank  Zone ID    Trip Count
1     237      408,760
2     132      403,668
3     236      363,939
4     161      361,461
5     186      284,050
6     162      277,355
7     142      253,015
8     138      245,319
9     230      245,043
10    163      217,202

=== BOTTOM 10 PICKUP ZONES BY TRIP COUNT ===
Rank  Zone ID    Trip Count
1     245        1
2     204        1
3      5         2
4     105        2
5     44         3
6     99         4
7     84         5
8     115        5
9     199        6
10    156        7

Total distinct pickup zones: 262
Total trips across all zones: 7,665,247

```

Figure A2. Detailed HDFS output inspection evidence.

B Reproducibility Checklist

Before submitting or demonstrating the project, the following items can be checked:

1. Hadoop services are running and YARN is available.
2. The three cleaned CSV files are present in `/taxi_project/input/cleaned`.
3. The required mapper and reducer files are available from the project repository.
4. Any previous output directory is removed before rerunning the same job.
5. The Python command used by Hadoop Streaming is available to the task environment.
6. The final result is read from the reducer output in HDFS.

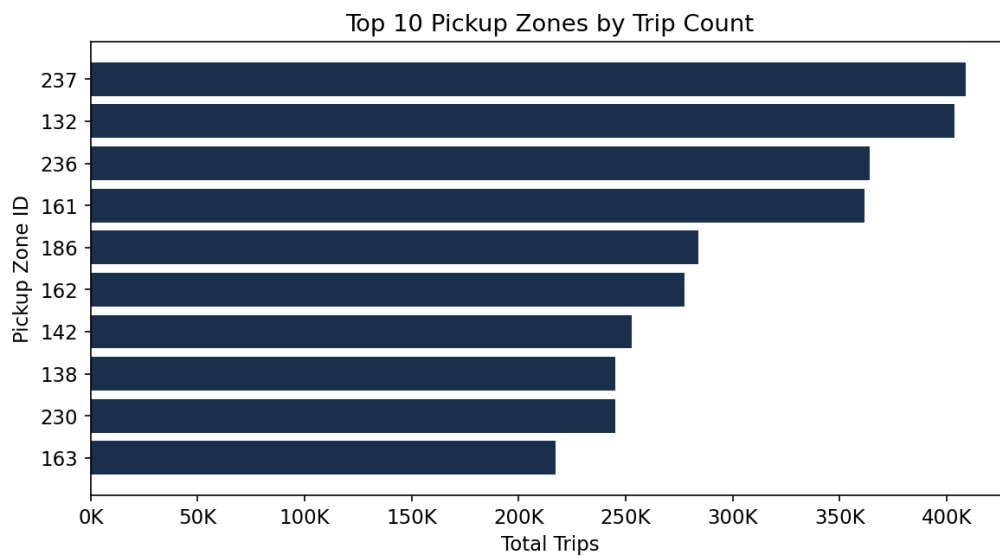


Figure A3. Additional command-line evidence from the analysis workflow.

C Code Availability

The report intentionally summarizes the implementation rather than filling many pages with source code. The complete mapper and reducer programs are available for inspection and execution at the following repository:

<https://github.com/erneste-uwimpuhwe/BIG-DATA-ESSENTIAL-INDIVIDUAL-ASSIGNMENT>

Keeping the code in a repository improves traceability, version control and reproducibility. The report explains the analytical logic and presents the evidence, while the repository provides the exact implementation files.

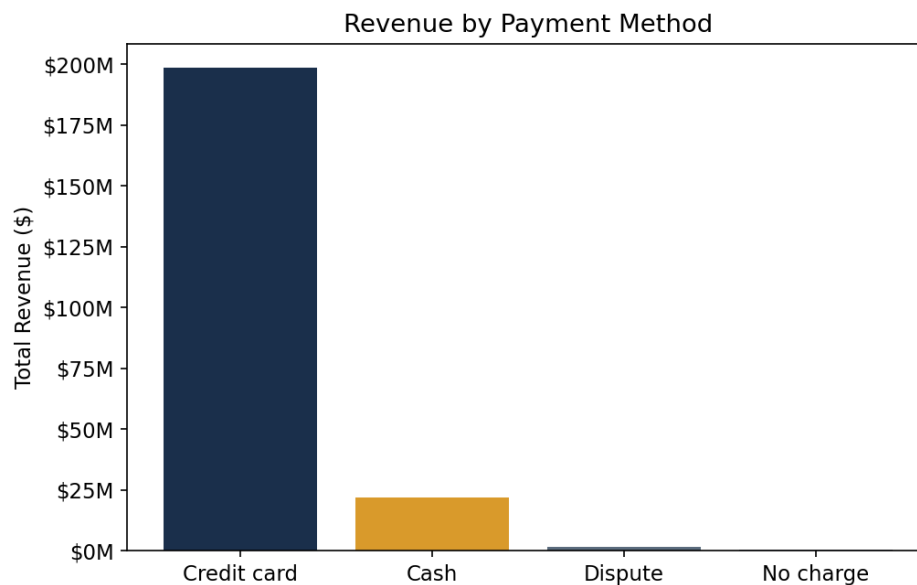


Figure A4. Additional result evidence supporting the implementation.

References

1. New York City Taxi and Limousine Commission. Trip Record Data. <https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page>
2. Apache Hadoop. Hadoop documentation and project resources. <https://hadoop.apache.org/>
3. Project source code repository. <https://github.com/erneste-uwimpuhwe/BIG-DATA-ESSENTIAL-INDIVIDUAL-ASSIGNMENT>